Design, Modeling and Real Time Implementation of a Two-Wheeled Self-Balancing Robot

Govind Gautam
ECE
KIET Group of Institutions, Ghaziabad
gautamgovind73@gmail.com

Khushi Nagar
ECE
KIET Group of Institutions, Ghaziabad
khushin862@gmail.com

Kritika Srivastava
ECE
KIET Group of Institutions, Ghaziabad
kritikasri444@gmail.com

Vibhuti Sharma
ECE
KIET Group of Institutions, Ghaziabad
vibhutis368@gmail.com

Richa Srivastava
Associate Professor
ECE
KIET Group of Institutions, Ghaziabad
richa.srivastava@kiet.edu

Manoranjan Kumar
Assistant Professor
ECE
KIET Group of Institutions, Ghaziabad
manoranjan.kumar@kiet.edu

***Abstract*—A self-balancing robot is capable to achieve dominant flexibility due to their compact size and have significant potential across many real time applications such as education and research. Due to real time control it can also be used in embedded system and sensor- fusion methods. However, by increasing their size it can be used as personal transport tool. We propose the design, modelling and implementation of a self-balancing robot developed by Arduino Nano and HC-05 Bluetooth module. The proposed work included descriptions of hardware with control logic, system architecture, PID controller design and tuning, and integration between hardware and software strategies. We used Wokwi for simulation of robot electronic circuit, Easy EDA for schematic and PCB design, and Fritzing for circuit design and wiring connection. The proposed work are also useful for human-robot interaction, self-navigation and mobile robotics because of low cost and affluent nonlinear dynamics.**

***Index Terms*—Self-balancing robot, Arduino Nano, MPU-6050, PID Controller, Stepper Motors, A4988, Embedded Systems**

I. Introduction

Self-balancing robots operate on the principle of maintaining the center of gravity above the wheelbase, similar to how humans subconsciously shift their feet to maintain balance while standing [1]. In such systems, even the slightest tilt results in momentary instability, requiring the control system to respond instantly. Because the robot constantly oscillates around the equilibrium point, advanced real-time feedback control algorithms such as PID, LQR, or fuzzy logic are often used [2], [3].

In modern robotics education, self-balancing robots serve as an important experimental platform because they allow students to practically observe how small changes in parameters (like PID values) can significantly alter the system's behavior [7], [8]. These robots works on concepts like closed-loop,noise filtering and sensor-drift compensation.

Therefore,a self balancing robot is seen as a foundational model regarding highly advanced systems which further brings us to some topics like humanoid robots,autonomous delivery vehicles,medical balance assist devices and mobility tools such as segway and hoverboards [6].

The main objective of this research is not only to build and further test the self-balancing robot but it is also to help us deeply understand each subsystem, which explains the reasoning behind certain design choices and present technical documentation [5], [17].

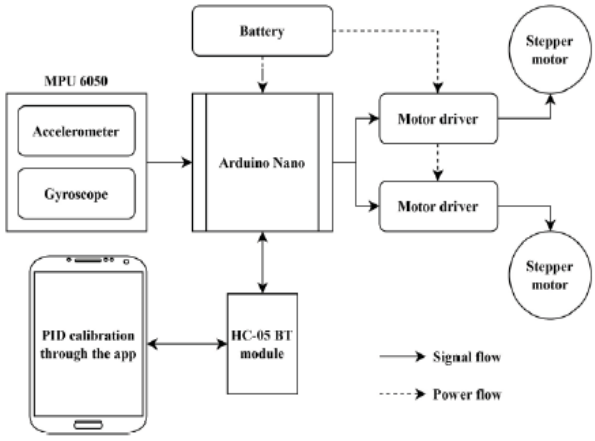


Fig. 1. Block Diagram of the Self-Balancing Robot

II. Literature Review

Self-balancing systems have been studied among many control approaches which includes PID,LQR,fuzzy logic and neural controllers, Previously DC motors with encoders were used for feedback measurement.Due to the miniaturization and low cost of the IMU- based estimation it became very popular.

Some challenges like gyroscope drift,accelerometer noise, motor non-linearity and some delay errors were highlighted previously.Whereas classic PID was outperformed by fuzzy

979-8-3315-4668-7/26/$31.00 © 2026 IEEE

and neural PID in dynamic environments but somehow classical PID remains ideal for cheap systems such as arduino-based robots ,Thus the project mainly focuses on accessibility ,clarity and reproductibility

III. Constituting Materials

To maintain dynamic stability the electronic and mechanical component work together for the construction of self-balancing robot. Each component contributes differently to structural, sensing support mathematics computation and accuracy of the system Understanding their working principles and limitation are essential for optimizing balance accuracy, reducing delay, and improving overall system response. [1], [7], [14]. [1], [7], [14].

A. Arduino Nano Microcontroller

The Arduino Nano is act like computational brain of a robot. The microcontroller names as ATmega328P microcontroller having ability of clocking at 16 MHz, that provide a compact and cost effective solution for self-balancing robot application. The benefits of Arduino nano is PID control, and motor actuation need to happen within milliseconds.

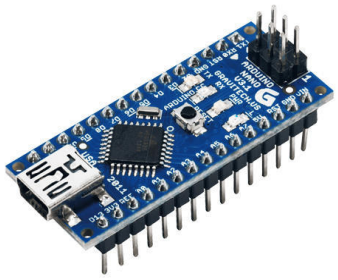


Fig. 2. Arduino Nano

Advantages of Arduino Nano for balancing control include Its ideality of for space and low power consumption, for troubleshooting , simple programming suitable for everyone And easy to understand and enough I/o pins for connecting IMU sensors, motor drivers and Bluetooth.

For PID consumption and complementary filtering the Arduino Nano's 16 MHz clock is used and for hardware timers provides adequate loop-time performance. The microcontroller's 2KB SRAM is sufficient for Storing sensor data and PID variables , while 32 KB flash memory is used for control algorithm codes [14].

B. MPU-6050 IMU Sensor

The combination of 3-axis gyroscope and 3-axis accelerometer in a one integrated module is the MPU-6050 is a 6-axis Inertial Measurement Unit .the angular velocity is measured by the gyroscope but accumulates drift over time. To overcome these individual weaknesses, a complementary filter is used:

TABLE I
Pin Description of Arduino Nano

Arduino Pin	Conn. Pin	Description	Component
A0 (Analog In)	RS	Register Select	LCD
A1 (Analog In)	E	Enable	LCD
A2 (Analog In)	D4	Data pin	LCD
A3 (Analog In)	D5	Data pin	LCD
A4 (Analog In)	D6	Data pin	LCD
A5 (Analog In)	D7	Data pin	LCD
0 (RX)	TX	Transmitter pin	Bluetooth
1 (TX)	RX	Receiver pin	Bluetooth
2	TRIG	Sends ultrasonic pulse	Ultrasonic
3	ECHO	Receives ultrasonic pulse	Ultrasonic
7	RELAY	Controls relay (NO pin)	Relay
9 (PWM)	BLUE	PWM→Blue LED	RGB LED
10 (PWM)	RED	PWM→Red LED	RGB LED
11 (PWM)	GREEN	PWM→Green LED	RGB LED

$$\theta = 0.98(\theta_{prev} + \omega\Delta t) + 0.02(\theta_{acc}) \quad (1)$$

This filter enables fast and efficient reponse gyro data with stable and long term accelerometer readings and a smooth and reliable tilt angle estimate [4], [11].

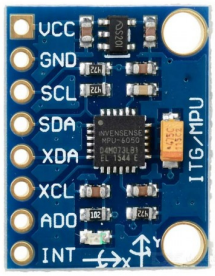


Fig. 3. MPU-6050

Because of its low-cost, availability, and proven performance in dynamic motion environment we use the MPU6050 in Self-balancing robot, additionally we use 12C communication up to high frequency, its digital processor [11].

TABLE II
Pin Description of MPU-6050

Type	Pins	Description
Power	VCC, GND	VCC is 5V input. GND is for grounding.
Communication	SDA, SCL	SDA transfers data; SCL provides clock for communication.
Auxiliary Comm.	XDA, XCL	Used to interface other modules when SDA/SCL lines are limited.
Other	AD0, INT	AD0 sets the LSB of the slave address for multiple MPU-6050 modules. INT indicates data available for MCU.

C. Stepper Motors

For precise stepwise movement and having ability to produce high holding torque which is necessary for stability [10] we use steppers motor. We don't use DC motors as stepper motor rotate in fixed increments which provide predictability

and stability to the robot during balancing movements. It's also used for static movements which do not require any feedback encoders , simplify the design .

However, stepper motors require higher current and may lose torque at high RPM, but their precision outweighs these drawbacks in balanc- ing applications [10]. Their linear torque profile makes them suitable for robots requiring frequent directional changes [17].

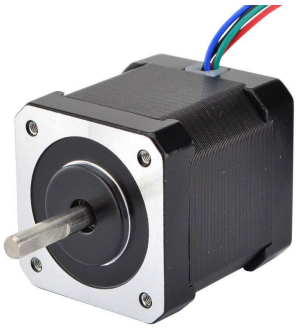


Fig. 4. The stepper motor

D. A4988 Stepper Motor Driver

The A4988 stepper motor driver works using STEP and DIR input signals to generate controlled current pulses to stepper motor [10]. STEP stands for step command, and DIR stands for direction pin. Every rising edge of the STEP pin commands the driver to move one microstep (full, 1/2, 1/4, 1/8, 1/16) speed of motor is defined by the frequency of STEP pulses, faster pulses higher RPM, slower pulses lower RPM. On the other hand, the direction command decides the direction in which motor rotates based on HIGH (clockwise) or LOW (counter-clockwise) logic level.

Besides micro-stepping support, features of A4988 stepper motor driver include support for bipolar stepper motor, it can control bipolar stepper motors up to 2A per coil. It also has built-in translator which converts STEP and DIR signals directly into coil control. Overcurrent Protection, Thermal shutdown protection, Under-Voltage Lockout (UVLO) . Usually, high current is drawn by any stepper motor but the A4988 limits this current to a safe value thus prevents overheating [18]. The A4988 due to its simplicity and rugged performance [10].

E. Mechanical Chassis

A mechanical chassis is typically a structural frame of a robot, machine, or vehicle [7]. It is a physical framework that provides support, shape, strength, mounting points, shape, and protection.

Important parameters to take into account are center of gravity and mass distribution, placing center of gravity close to the vertical center plane ensures stability, and low centralized mass helps in reducing torque. Moment of Inertia, keeping all

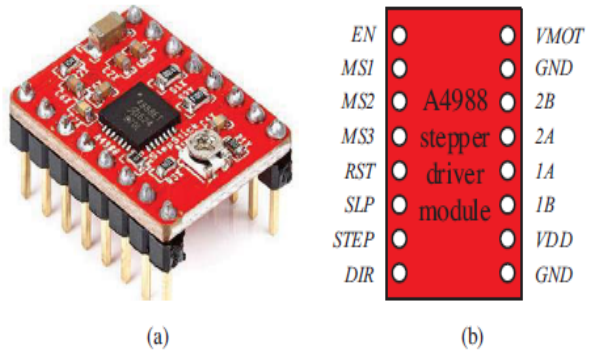


Fig. 5. (a) A4988 Stepper motor driver module, and (b) its pinout diagram

the mass concentrated near the vertical centerline reduces yaw inertia. [7].

F. HC-05 Bluetooth Module

The HC-05 is a serial communication bluetooth module used to make devices communicate wirelessly using Bluetooth Classic (SPP-Serial Port Protocol). Its role is to provide real-time tuning of PID parameters. Sensor readings are send from the microcontroller to HC-05 bluetooth module through UART and live monitoring of IMU data is achieved. [5].

SPP (Serial Port Profile) is the core fundamental of HC-05 bluetooth module through which it behaves like a wireless serial cable. It reduces the burden of connecting Arduino to laptop with a USB wire, the HC-05 module sends the exact data wirelessly. SPP protocol enables full-duplex communication with standard UART pins of the Arduino Nano. High baud rates upto 1,382,400 bps is supported, although common configurations are between 9600 and 38400 bps [14].

One key benefit of using HC-05 bluetooth module is that it can operate in both Master and Slave modes. This versatility of this wirleless module allows it to start or receive connections based on system's requirement. This 2-wheel self-balancing robot set up in slave mode, in slave mode no scanning, pairing, or initiating the communication is needed, this slave mode ensures automatic pairing and also send command for tuning Kp, Ki, Kd values [5].

The HC-05 operates using the Bluetooth SPP (Serial Port Profile) protocol, enabling full-duplex communication with standard UART pins of the Arduino Nano. It supports baud rates up to 1382400 bps, although common configurations in robotics applications range between 9600 and 38400 bps, ensuring stable and low-latency data transfer [14]. The class-2 transmission range of approximately 10 meters is sufficient for this bluetooth module for indoor laboratory experiments.

G. Miscellaneous Components

Apart from the main electronic devices, several other supporting components are used to ensure predictable performance, reliable power distribution, and safe mechanical assembly of 2-wheel self-balancing robot [7].

TABLE III
PIN DESCRIPTION OF HC-05 BLUETOOTH MODULE

Type	Pins	Description
Power	VCC, GND	VCC is 5V input. GND is for grounding.
Key	EN (Enable)	Used to toggle between Data mode and AT command mode. Set high to enter AT mode.
Serial	RX, TX	RX is the receiver; TX is the transmitter. Used for serial communication.
State	State	Connected to the module's inbuilt LED and used as feedback to check module status.

A 12V Li-ion battery is used to power the stepper motor and motor drivers. Its lightweight architecture and high power density make is a good match for robotic application [6]. For delivering clean and regulated power, voltage regulator and buck converters are used. For avoiding IMU noise stable power is crucial [14].

Toggle switches, JST connectors, and power port provide battery a safe disconnection and organized cable routing. Mechanical Chassis fasteners and structural elements including nuts, bolts, spacers, acrylic plates, and motor brackets maintain precise mechanical positioning and robustness. Accurate component alignment helps mitigate vibrations in IMU, elevating angle accuracy.

H. Supporting Tools

An assortment of tools and platform was used for circuit designing, simulation, and implementation of this robot, and these are:

Wokwi was used for simulating the circuit for the self balancing robot and virtually simulating the Arduino code. This is the web-based simulation platform that is used for testing sensor integration and control logic before software implementation with the purpose of reducing development time and attenuating hardware debugging issues.

EasyEDA a tool providing professional environment for creating circuit diagrams,it was used for schematic design and PCB layout to visualize the connections among components. EasyEda,is also used for planning the physical layout of electronic components,by which proper signal routing and power distribution can be ensured.

Fritzing The main purpose of using Fritzing was for creating circular designs and comprehensive visualizations of wiring connections. The breadboard view feature of Fritzing made it really simple for understanding the the physical wiring layout, which proved to be very helpful during the assembly phase and for documents purposes.

IV. WORKING OF THE SELF-BALANCING ROBOT

This self-balancing robot is built on the concept of an inverted pendulum, where the center of gravity and the mass evenly distributed above the wheel axis. The tilt angle of the robot is continuously sensed by the accelerometer (MPU-6050) and the same is sent to the Arduino Nano [11].

The measured tilt is compared by microcontroller with the desired upright position and the error is then calculate by finding the difference between them. This calculated error is then processed by a PID controller and a corrective output is then generated to move the wheels forward or backward [4].

When the robot is leaned forward,is being brought back to balance by the wheels.When the robot leans backward,the movement of wheels occur in the opposite direction [1]. The PID output is converted into step pulses for the stepper motors by A4988 drivers. This enables the smooth and controlled movement of the wheels [20]. The continuous cycle which includes sensing, filtering,error correction and actuation keeps the robot upright and stable in real time [10].

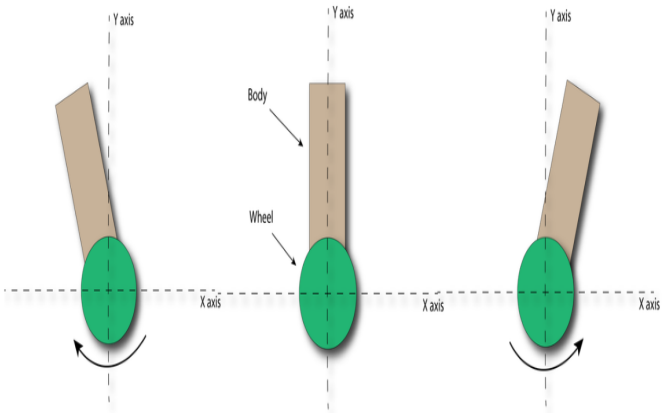


Fig. 6. WORKING PRINCIPLE

V. PID CONTROLLER: THEORETICAL BACKGROUND

The Proportional-Integral-Derivative (PID) controller is used most frequently as a closed-loop control algorithm in the robotics and in the industrial automation.It generates a corrective control signal which further minimizes the error between the desired setpoint and the measured process. In the self-balancing robot the PID controller is used to maintain the robot's upright position which is done by adjusting its wheel movement [2], [15].

The robot thus tries everytime to keep its tilt angle at zero degrees because any deviation which occurs here can cause an error,After this the PID controller calculates how much corrective motion is needed to cancel the tilt and restores balance [1]. Its fast response and its simplicity makes the PID highly effective for all the dynamic systems which are inverted pendulums, UAVs, motor drives and the two-wheeled balancing robots [16], [17].

A. PID Control Law

The PID controller computes the control output $u(t)$ using three components:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\,d\tau + K_d \frac{de(t)}{dt} \qquad (2)$$

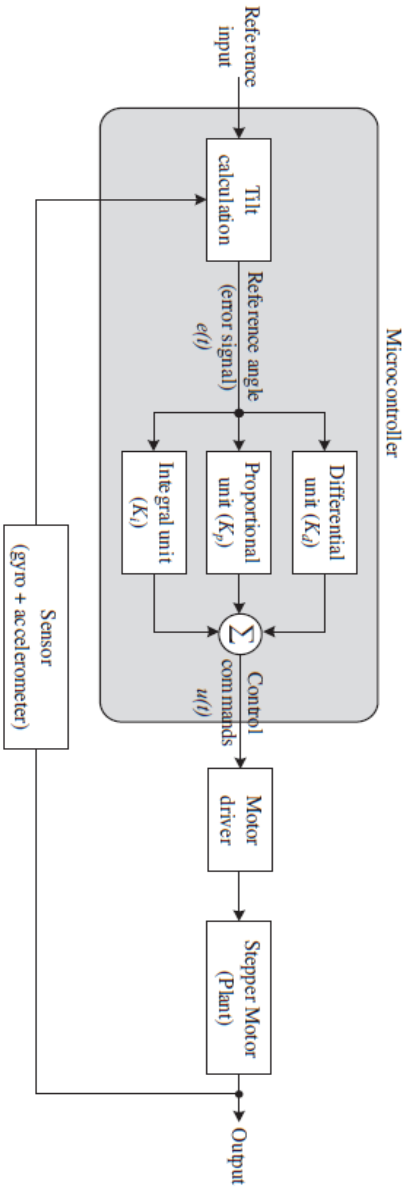


Fig. 7. The closed loop control system of the self-balancing robot

here $e(t)$ is the instantaneous error, K_p is the proportional gain, K_i is the integral gain, and K_d is the derivative gain. Each term contributes itself differently to the system stability, its speed of response, and its oscillation behavior [20].

B. Proportional Control (P-Term)

This Proportional term generates a control action which is directly proportional to the current error,The larger value of K_p increases the system responsiveness as it applies stronger corrective motion.However, excessive K_p somehow causes the oscillations and the overshoot which results in making the robot wobble and move aggressively [1]. The specific or exact tuning of the proportional gain is critical for quick and stable correction [15].

C. Integral Control (I-Term)

The Integral term accumulates the past errors over time which then eliminates or reduces the steady-state errors, As it's a very important step for the balancing robots because slight sensor offsets, mechanical bias or uneven weight distribution causes a constant tilt in one direction which is not a very favorable condition [6]. The I-term neutralizes these biases but too much integral gain then results in the slow oscillation,in the drift or even in the delayed recovery after the disturbance [15].

D. Derivative Control (D-Term)

Based on its rate of change ,The Derivative Term predicts the future errors behaviour ,It further stabilizes the fast tilt variations and reduces the overshooting by acting like a damping force [20]. In the balancing robots the D-term is very essential or is very important as it controls vibrations, sudden pushes and also rapid oscillations which are mostly caused by the motor inertia and the sensor noise [7]. The Proper derivative tuning increases the stability and minimizes the wobbling.

E. Need for PID in Self-Balancing Robots

Self-balancing robots are inherently unstable systems that behave like inverted pendulums. Their center of mass is above the wheel axis, which means even slight tilts must be corrected immediately. PID control is preferred because it is simple yet effective for real-time balancing [1], provides fast corrective action for tilt errors, works well with noisy IMU signals when paired with filtering [4], compensates for motor lag, friction, and uneven load, and stabilizes both small and large disturbances. The efficiency of the balancing robot heavily depends on correctly tuned PID values [1], [15].

TABLE IV
PID CONTROLLER PARAMETERS AND THEIR EFFECTS

Parameter	Symbol	Effect
Proportional Gain	K_p	Increases response to error. Higher values reduce tilt quickly but may cause oscillations.
Integral Gain	K_i	Eliminates long-term drift and steady-state errors. Too high causes overshoot.
Derivative Gain	K_d	Adds damping by reacting to rate of change. Reduces overshoot but increases noise sensitivity.

F. PID Tuning Challenges

Tuning PID manually is time-consuming because MPU6050 introduces noise and drift [11], motor nonlinearity affects speed response [10], voltage drops change motor torque [7], high K_p causes oscillations, high K_i causes slow drifting, and high K_d introduces jitter from sensor noise. Therefore, real-time tuning using a Bluetooth-based mobile application

significantly improves accuracy and reduces development time [5], [14].

VI. Hardware Implementation

The hardware implementation of the self-balancing robot focuses on correct placement, wiring, and power management of all components so that the control system can respond quickly and accurately. Since a balancing robot behaves like an inverted pendulum, even small mechanical or electrical imperfections can lead to drift, oscillations, or delayed correction [**?**]

The MPU-6050 IMU is mounted at the geometrical center of the chassis to obtain unbiased tilt measurements. Any displacement from the center causes additional rotational forces that affect angle accuracy. To reduce mechanical vibrations transferred from the motors, a thin foam layer is placed under the sensor module, improving the stability of accelerometer and gyroscope readings [11].

Both stepper motors are fixed on opposite sides of the chassis using rigid brackets so that equal torque is applied on each wheel. Proper alignment ensures that the robot does not pull to one direction during balancing. The motors are connected to A4988 drivers, which receive STEP and DIR signals from the microcontroller. Microstepping configuration is set to 1/8 or 1/16 step to achieve smoother wheel motion and reduce jerky corrections during balancing [10]. The current limit on each driver is adjusted to provide adequate torque without overheating.

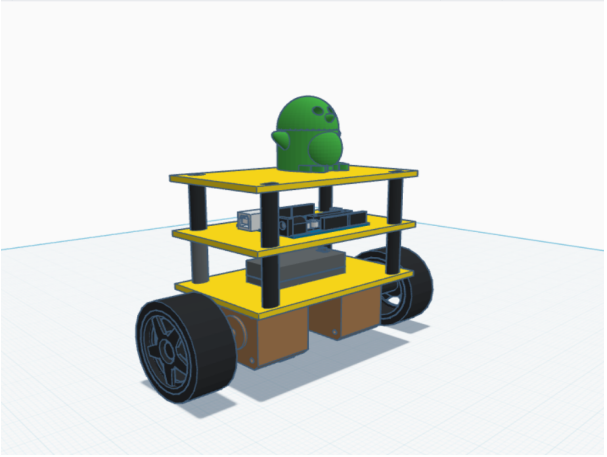


Fig. 8. 3D Model

VII. Software Simulation

Before physical implementation, the self-balancing robot system was simulated in software to validate the control algorithms and predict system behavior under various conditions [4], [12]. MATLAB Simulink was used to model the dynamics of the inverted pendulum, including motor response, PID control, sensor noise, and delay effects.

The simulation allowed testing different PID parameters, observing tilt response, and analyzing stability without risking hardware damage. This step is crucial in understanding how the robot reacts to disturbances, sudden tilts, or variations in system parameters. It also helps identify potential issues such as oscillations, overshoot, or slow response before hardware integration [12].

The model included:

- Inverted pendulum dynamics to represent the robot body.
- PID controller block to correct tilt errors.
- Sensor noise and delay simulation for MPU-6050 readings.
- Motor dynamics to emulate stepper motor behavior.

Software simulation results demonstrated that correct PID tuning significantly improved stability, minimized oscillations, and ensured smooth wheel movements. The simulation output guided the initial selection of K_p, K_i, and K_d values for real-world testing, reducing trial-and-error iterations on hardware [12].

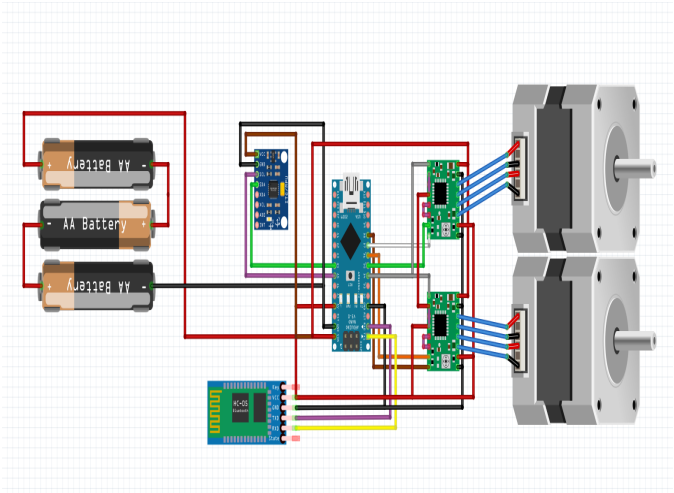


Fig. 9. Circuit Diagram

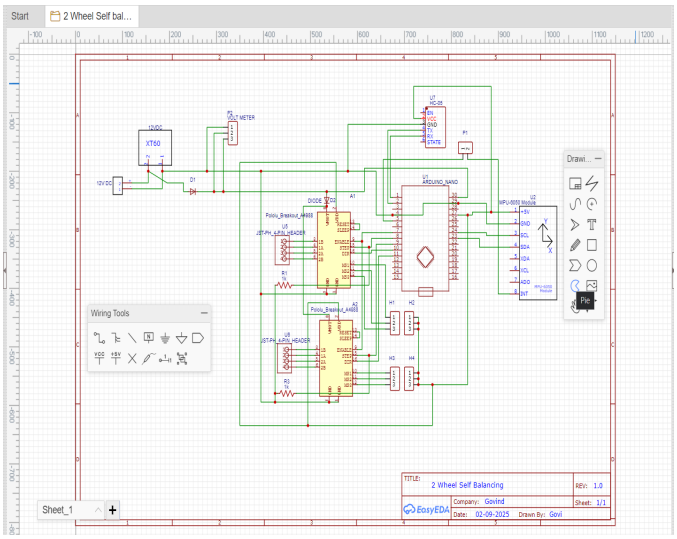


Fig. 10. Schematic of Circuit

VIII. Limitations

In spite of being a reliable robot for academic as well as practical purposes, the self balancing robot has many drawbacks which effects its accuracy. The Gyro sensor MPU-6050 is more prone to electrical noise, vibration and some surface change, which introduces error in angle estimation [11]. When the robot is moving on a rough terrain or at a higher speed these kinds of error become more to a number and can reduce the stability of the control loop to a great extinct.

The steppers motor, although they are error free in the calculation, shows some torque reduction at higher RPM which can produce vibrations. These problems reduces the maximum speed and therefore limit the ability to recover from large tilted angles [10]. The A4988 motor driver also drawbacks the advantage to current shaping as compared to modern drivers, which leads to addition of noise.

The variation in battery voltage also plays a major role in the change in performance. As the battery voltage decreases, the torque decreases leading to slow speed and vibrations [7]. Despite of having high processing speed of Aurdino Nano, the robot still faces limitations like bluetooth communication and motor control at the same time [14]. Mechanical errors like wheel position, asymmetry leads to consistent generation of errors in robot's motion [7].

IX. Future Scope

We can now basically conclude how several new improvements can enhance the overall accuracy and the capability of these robot.Also in order to reduce the drift and also to improve the tilt estimation under heavy vibrations, an extended Kalman fibre or a Madgwick fiber can be installed [4]. Some filtering techniques like these helps these robots to operate accurately even on uneven surfaces. We can then upgrade the drive mechanism to BLDV motors which have encoder feedback that would offer smoother control, higher torque and better response time when compared to the stepper motors [18]. Also,TMC series motor drivers can limit or reduce noise ,they can increase smoothness and also provides the system with more efficient current control.

We can make certain changes or advancements in order to make the robot move on its own and to avoid obstacles by adding cameras, ultrasonic sensors or even LIDAR.With the help of SLAM(Simultaneous Localization and Mapping) the robot can create a map of the indoor spaces and can navigate or move around them. [19] IoT features also helps cloud data storage and many more systems like remote monitoring,Machine learning is used to automatically adjust the PID settings,in order to make the robot adapt to different surfaces,loads,tilts or different variations.

X. Conclusion

This project concludes of a simple,clear and practical implementation of a cheap but budget-friendly self - balancing robot.This project is perfect for the purpose of academic learning and it also provides understanding of the control system and the embedded system in a proper manner.

References

[1] J. Mathew, "Inverted Pendulum Modelling and Stabilization Techniques," Dynamics and Control Journal, 2022.
[2] S. Yuan, "Dynamic Modelling and Control of Two-Wheeled Balancing Robots Using Lagrangian Approach," 2016.
[3] J. Wu, "Fuzzy Logic Based Control for Self-Balancing Robots," International Journal of Automation, 2011.
[4] H. Lin, "Kalman and Complementary Filtering for Robotics Applications," Robotics Systems Journal, 2024.
[5] HowToMechatronics, "Self-Balancing Robot Using Arduino," 2022.
[6] R. Goel, "IMU Drift, Vibration Issues, and Balancing Performance," Sensors and Actuators Journal, 2023.
[7] A. Malik, "Mechanical Stability and Chassis Optimization for Educational Robots," 2023.
[8] S. Nayak, "Robotics Education Platforms and Their Effectiveness," Education in Engineering Journal, 2024.
[9] V. Rao, "Trajectory Stability Analysis in Balancing Mobile Robots," 2021.
[10] Allegro Microsystems, "A4988 Stepper Motor Driver: Technical Datasheet," 2022.
[11] InvenSense, "MPU-6050: Integrated 6-Axis Motion Sensor Datasheet," 2021.
[12] P. Tipsuwanporn, "State-Feedback Control Techniques for Balancing Robots," Control Systems Review, 2020.
[13] T. Mahowik, "Self-Balancing Robot using Nintendo Wii Accelerometer," 2020.
[14] R. Arya, "Embedded Control Design Using ESP32 and Arduino Platforms," Journal of Embedded Engineering, 2023.
[15] M. Patel, "Hybrid Fuzzy-PID Control for Real-Time Balancing Systems," Control Theory Letters, 2022.
[16] P. Singh, "Practical Applications of Classical Control Theory," Academic Press, 2024.
[17] J. Borja, "Low-Cost Self-Balancing Robot Design and Analysis," 2020.
[18] IJERT, "Precision Robotics and Real-Time Motor Control," IJERT Research Journal, 2025.
[19] L. Zhang, "Neural Adaptive Controllers for Balancing Robots," Advanced Robotics Journal, 2025.
[20] J. Ziegler and N. Nichols, "Optimal PID Tuning Methods," ASME Transactions, 1942.
[21] K. Ogata, "Modern Control Engineering," 6th Edition, Pearson, 2010.
[22] S. Thrun, W. Burgard, D. Fox, "Probabilistic Robotics," MIT Press, 2005.
[23] M. Negnevitsky, "Artificial Intelligence: A Guide to Intelligent Systems," 4th Edition, Pearson, 2011.
[24] R. Siegwart, I. Nourbakhsh, "Introduction to Autonomous Mobile Robots," MIT Press, 2011.
[25] B. Siciliano, L. Sciavicco, "Robotics: Modelling, Planning and Control," Springer, 2010.
[26] Arduino Project Hub, "Self-Balancing Robot Projects," 2023.
[27] A. Mahony, "Complementary Filter Design for IMU Applications," IEEE Trans. Robotics, 2008.
[28] J. Zhang, "Kalman Filtering for Nonlinear Systems in Robotics," Int. Journal of Robotics Research, 2019.
[29] H. Bai, "Stepper Motor Microstepping Control Techniques," IEEE Robotics Letters, 2022.
[30] S. Kim, "MATLAB Simulink Modeling of Self-Balancing Robots," Simulation Modelling Practice and Theory, 2021.
[31] R. Mahajan, "IoT-Based Control of Mobile Robots," Sensors Journal, 2024.
[32] L. Costa, "PID Control Applications in Robotics," International Conference on Robotics, 2023.
[33] P. Huang, "Hybrid PID-Fuzzy Controllers for Real-Time Balancing," IEEE Transactions on Control Systems Technology, 2020.
[34] T. Keller, "IMU Noise Reduction Using Complementary and Kalman Filters," IEEE Access, 2022.